

Untitled diff

81 removals

103 lines

160 additions

155 lines

```

1 | class TaskQueue:
2 |     # Initialize a bounded first-in, first-
    out (FIFO) task queue.
3 |     # max_size defines the maximum number
    of tasks that can be stored
4 |     # before new enqueue operations are
    rejected.
5 |
6 |     def __init__(self, max_size=100):
7 |         self.queue = [] # Internal
    storage for queued tasks in arrival order.
8 |         self.max = max_size # Maximum
    allowed queue length.
9 |
10 |     # Add a task to the end of the queue.
11 |
12 |     # Returns:
13 |     # True - task was successfully
    added.
14 |     # False - queue is already at
    capacity.
15 |
16 |     def enqueue(self, task):
17 |         if len(self.queue) >= self.max:

```

```

1 | class TaskQueue:
2 |     # Create a bounded first-in, first-out
    (FIFO) queue for task management.
3 |     #
4 |     # Tasks are stored in the order they
    are received and processed in the
5 |     # same order. The queue enforces a
    maximum capacity to prevent unbounded
6 |     # growth and provides simple operations
    for adding, removing, and
7 |     # inspecting queued work.
8 |     #
9 |     # Args:
10 |     # max_size (int): Maximum number of
    tasks that may be stored in the
11 |     # queue at one time. Additional
    enqueue attempts are rejected
12 |     # once this limit is reached.
13 |
14 |     def __init__(self, max_size=100):
15 |         self.queue = [] # Ordered
    collection of pending tasks.
16 |         self.max = max_size # Upper
    bound on the number of queued tasks.
17 |
18 |     # Add a task to the back of the queue.
19 |     #
20 |     # The task is appended only if
    sufficient capacity remains.
21 |     #
22 |     # Args:
23 |     # task: Object representing the
    work item to be queued.
24 |     #
25 |     # Returns:
26 |     # bool:
27 |     # True if the task was
    successfully queued.
28 |     # False if the queue is already
    full.
29 |
30 |     def enqueue(self, task):
31 |         if len(self.queue) >= self.max:

```

```
15         return False         # Reject
    new tasks when capacity is reached.
```

```
16         self.queue.append(task)
17         return True
18
```

```
19         # Remove and return the task at the
    front of the queue.
```

```
20         # Returns:
```

```
21         # The oldest queued task if one
    exists.
```

```
22         # None if the queue is empty.
```

```
23         def dequeue(self):
24             if not self.queue:
```

```
25                 return None         # No tasks
    available to process.
```

```
26         return self.queue.pop(0)
27
```

```
28         # Determine whether the queue currently
    contains any tasks.
```

```
29         # Returns:
```

```
30         # True if the queue is empty,
    otherwise False.
```

```
31         def is_empty(self):
32             return len(self.queue) == 0
33
```

```
34         # Return the current number of tasks
    waiting in the queue.
```

```
35         def size(self):
36             return len(self.queue)
37
```

```
38         # Retrieve the next task scheduled for
    processing without removing it.
```

```
30         return False         # Capacity
    reached; reject the new task.
```

```
31
32         self.queue.append(task)
33         return True
34
```

```
35         # Remove and return the next task to be
    processed.
```

```
36         #
```

```
37         # Because the queue follows FIFO
    semantics, the oldest queued task is
```

```
38         # always returned first.
```

```
39         #
```

```
40         # Returns:
```

```
41         # The task at the front of the
    queue, or None if no tasks are
```

```
42         # currently waiting.
```

```
43         def dequeue(self):
44             if not self.queue:
```

```
45                 return None         # No
    pending tasks are available.
```

```
46
47         return self.queue.pop(0)
48
```

```
49         # Check whether the queue currently
    contains any tasks.
```

```
50         #
```

```
51         # Returns:
```

```
52         # bool: True if the queue is empty,
    otherwise False.
```

```
53         def is_empty(self):
54             return len(self.queue) == 0
55
```

```
56         # Get the current number of queued
    tasks.
```

```
57         #
```

```
58         # Returns:
```

```
59         # int: Total number of tasks
    waiting in the queue.
```

```
60         def size(self):
61             return len(self.queue)
62
```

```
63         # Inspect the next task scheduled for
    processing without removing it.
```

```
64         #
```

```
65         # This operation allows callers to view
    the upcoming task while leaving
```

```
66         # the queue unchanged.
```

```
67         #
```

```

39 | # Returns:
40 | # The task at the front of the queue.
41 | # None if the queue is empty.
42 | def peek(self):
43 |     if self.queue:
44 |         return self.queue[0]

```

```

45 |         return None
46 |
47 |
48 | class RateLimiter:

```

```

49 |     # Initialize a simple sliding-window
    rate limiter.

```

```

50 |     #

```

```

51 |     # Parameters:

```

```

52 |     # limit - maximum number of allowed
    requests within the window.

```

```

53 |     # window - rolling time window length
    in seconds.

```

```

54 |     #

```

```

55 |     # The limiter stores timestamps for
    recent successful requests and

```

```

56 |     # evaluates each new request against
    the configured threshold.

```

```

57 |     def __init__(self, limit, window):

```

```

58 |         self.limit = limit # Maximum
    allowed requests per window.

```

```

59 |         self.window = window # Size of
    the rolling time window in seconds.

```

```

60 |         self.calls = [] #
    Timestamps of accepted requests.

```

```

61 |

```

```

62 |     # Determine whether a request at the
    given timestamp should be allowed.

```

```

63 |     #

```

```

64 |     # Expired timestamps that fall outside
    the current window are removed

```

```

65 |     # before evaluating the request count.

```

```

68 | # Returns:

```

```

69 | # The task at the front of the
    queue, or None if the queue is empty.

```

```

70 | def peek(self):
71 |     if self.queue:
72 |         return self.queue[0]

```

```

73 |
74 |         return None
75 |
76 |
77 | class RateLimiter:

```

```

78 |     # Implement a fixed-capacity sliding-
    window rate limiter.

```

```

79 |     #

```

```

80 |     # The limiter tracks timestamps of
    accepted requests and determines
81 |     # whether a new request can be allowed
    based on how many requests have
82 |     # occurred within the configured time
    window.

```

```

83 |     #

```

```

84 |     # Args:

```

```

85 |     # limit (int): Maximum number of
    requests permitted during a single

```

```

86 |     # rolling time window.

```

```

87 |     # window (int | float): Duration of
    the rolling window in seconds.

```

```

88 |     def __init__(self, limit, window):

```

```

89 |         self.limit = limit # Maximum
    requests allowed within the window.

```

```

90 |         self.window = window # Rolling
    time window length in seconds.

```

```

91 |         self.calls = [] #
    Timestamps for previously accepted
    requests.

```

```

92 |

```

```

93 |     # Evaluate whether a request should be
    allowed at the specified time.

```

```

94 |     #

```

```

95 |     # Requests that fall outside the active
    window are discarded before the

```

```

96 |     # current request is evaluated. If the
    number of remaining requests is

```

```

97 |     # below the configured limit, the
    request is accepted and its timestamp

```

```

98 |     # is recorded.

```

```

99 |     #

```

```

100 |    # Args:

```

```
66 #
67 # Returns:
```

```
68 # True - request is permitted and
    recorded.
69 # False - rate limit has been
    exceeded.
```

```
70 def allow(self, timestamp):
```

```
71     # Retain only timestamps that still
    fall within the active window.
72     self.calls = [t for t in self.calls
    if timestamp - t < self.window]
```

```
73
74     if len(self.calls) < self.limit:
75         self.calls.append(timestamp)
76         return True
77
```

```
78     return False # Request
    denied because the limit was reached.
```

```
79
80
81 def retry(func, attempts=3, exceptions=
    (Exception,)):
```

```
82     # Execute a callable and automatically
    retry on specified exceptions.
```

```
83 #
```

```
84 # Parameters:
```

```
85 # func - callable to execute.
```

```
86 # attempts - total number of
    execution attempts.
```

```
87 # exceptions - tuple of exception
    types that should trigger a retry.
```

```
88 #
```

```
101 # timestamp (int | float): Time
    associated with the incoming request.
```

```
102 #
103 # Returns:
```

```
104 # bool:
```

```
105 # True if the request is
    permitted.
```

```
106 # False if the rate limit has
    been reached.
```

```
107 def allow(self, timestamp):
```

```
108     # Remove request records that are
    no longer relevant to the current
109     # rolling window.
```

```
110     self.calls = [
111         t for t in self.calls
112         if timestamp - t < self.window
113     ]
```

```
114
115     if len(self.calls) < self.limit:
116         self.calls.append(timestamp)
117         return True
118
```

```
119     return False # Request
    exceeds the allowed request rate.
```

```
120
121
122 def retry(func, attempts=3, exceptions=
    (Exception,)):
```

```
123     # Execute a callable with automatic
    retry behavior.
```

```
124 #
```

```
125 # The target function is invoked
    repeatedly until it succeeds or the
```

```
126 # maximum number of attempts has been
    exhausted. Only exceptions whose
```

```
127 # types are included in the exceptions
    parameter trigger a retry.
```

```
128 #
```

```
129 # Args:
```

```
130 # func (callable): Function to
    execute.
```

```
131 # attempts (int): Total number of
    execution attempts, including the
```

```
132 # initial call.
```

```
133 # exceptions (tuple[type]):
    Exception types that should be caught and
```

```
134 # retried.
```

```
135 #
```

```

89 | # Returns:
90 | # The result produced by func() if
    | any attempt succeeds.
91 |
92 | # Raises:
93 | # The last caught exception after all
    | retry attempts are exhausted.
94 |     for i in range(attempts):
95 |
96 |         try:
97 |             return func() # Attempt
    | execution.
98 |         except exceptions as e:
99 |             if i == attempts - 1:
100 |
101 |                 raise # No
    | retries remain; propagate the exception.
102 |
103 |
104 | # Defensive fallback. This path is
    | normally unreachable because the
105 | # function either returns a result or
    | raises an exception.
106 | return None

```

```

136 | # Returns:
137 | # The value returned by func() if
    | an attempt succeeds.
138 |
139 | # Raises:
140 | # Any exception not listed in
    | exceptions.
141 | # The final caught exception if all
    | retry attempts fail.
142 | def retry(func, attempts=3, exceptions=
    | (Exception,)):
143 |     for i in range(attempts):
144 |
145 |         try:
146 |             return func() # Execute
    | the target operation.
147 |
148 |         except exceptions:
149 |
150 |             # Re-raise the exception
    | once the final retry has failed.
151 |             if i == attempts - 1:
152 |                 raise
153 |
154 |     # Defensive fallback. Under normal
    | execution this statement is never
155 | # reached because the function
    | either returns a result or raises an
156 | # exception before the loop

```